Atari game PONG에 강화학습 알고리즘 적용하기

강화학습실제

산업인공지능학과 2025254012 이은혜

목차

1. PPO 알고리즘
2. 구현 및 소스코드 설명
3. 학습결과
4. 실행결과
5. 프로그램 소스

[Proximal Policy Optimization]

Proximal Policy Optimization Algorithms

(Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017) arXiv preprint arXiv:1707.06347.)

- 기존의 TRPO(Trust Region Policy Optimization)에서 정책이 갑자기 크게 바뀌는 것을 막기 위한 신뢰영역 제약 조건을 훨씬 간단하게 구현한 방식
- Clipped Surrogate Objective를 사용해 정책이 과도하게 변하는 것을 방지함
- Policy Gradient Methods 방법
- 과거의 정책을 재사용하기위해 Importance Sampling을 사용함
- Advantage 추정은 GAE(Generalized Advantage Estimate)를 사용

* Clipped Surrogate Objective

PPO 알고리즘의 핵심 손실함수인 Clipped Surrogate Objective에 대하여

1. PPO의 정책 확률 비율

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

새로운 정책이 같은 (a | s)에 대해 이전 정책보다 행동을 얼마 더 자주 선택하는지를 나타낸다.
이 값이 1이라면 정책이 바뀌지 않은 것이고 1보다 크거나 작으면 바뀐 것이다.

2. 기존 Conservative Policy Iteration (CPI) 손실식

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right] = \hat{\mathbb{E}}_t\left[r_t(\theta)\hat{A}_t\right].$$

기존의 정책 경사 방식은 r_t가 커질수록 손실이 계속 증가하여 정책이 지나치게 많이 바뀌는 문제가 발생한다.

3. Clipped Objective

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right]$$

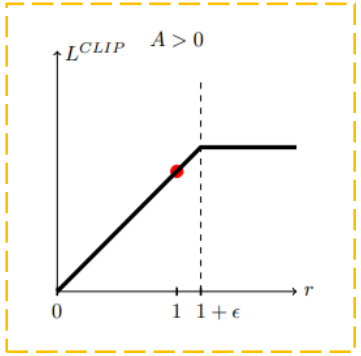
r_t가 클리핑 범위인 [1−ε, 1+ε]를 벗어나는 순간 손실은 더 이상 증가하지 않는다.
Advantage가 양수일때는 상한, 음수일 때는 하한으로 제한된다.

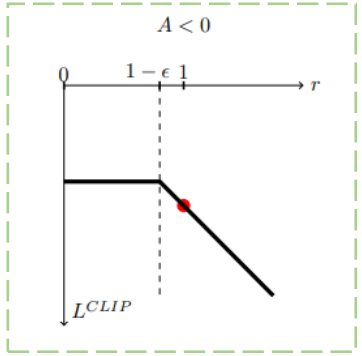
* Clipped Surrogate Objective

PPO 알고리즘의 핵심 손실함수인 Clipped Surrogate Objective에 대하여

A>0, A<0 에 따른 손실 함수 모양

확률 비율 r이 1에서 너무 멀어지면 손실이 더 이상 증가하거나 감소하지 않고 수평선으로 제한된다.

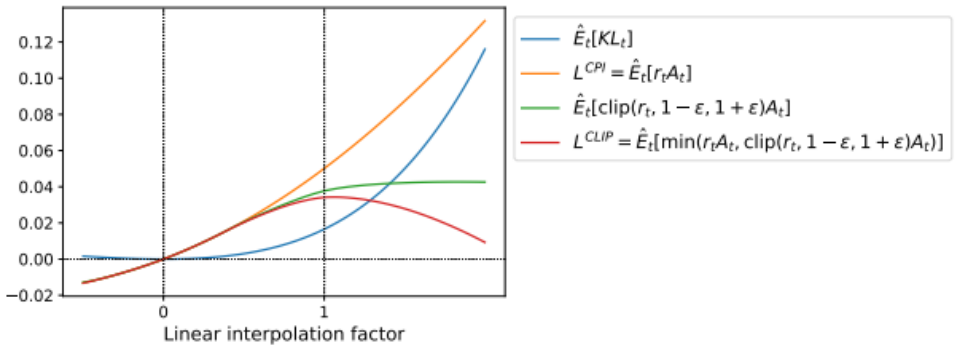




따라서 정책이 너무 급격히 바뀌는 것을 막을 수 있어 갑자기 큰 업데이트로 인한 성능저하, 불안정한 학습, 샘플 효율성 저하를 방지할 수 있다.

PPO Clipped Surrogate Objective의 효과

surrogate objective를 비교한 그래프로 주황색이 기존의 policy gradient 방식, 빨간색이 Clipped Objective 방식이다.



policy가 크게 바뀌면 기존 방법은 계속 증가하며 과도한 업데이트를 유도할 수 있다.
PPO Clipped Surrogate Objective는 일정 범위 이후에 손실이 줄어드는 방향으로 바뀌어 정책이 너무 많이 바뀌면 penalty를 준다.
이를 통해 업데이트를 억제하게 하여 정책을 안정적으로 개선할 수 있다.

1. PPO 알고리즘

* PPO 학습 알고리즘 구조

Algorithm 1 PPO, Actor-Critic Style

for iteration$=1, 2, \ldots$ **do**
  for actor$=1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Trajectory 수집	병렬 N개의 actor가 각 T step씩 rollout
Advantage 계산	GAE 사용하여 $\hat{A}_t$계산
Surrogate 최적화	Clipped loss 기반으로 여러 epoch 학습
Policy 업데이트	old policy ← new policy

PPO의 전체 학습 흐름은 매 iteration마다 여러 actor가 데이터를 수집하고
advantage를 계산한 뒤 Clipped Surrogate Loss를 여러 epoch에 걸쳐 minibatch 방식으로 최적화 함
PPO는 policy를 반복적으로 개선함

2. 구현 및 소스코드 설명

* PPO 학습 알고리즘 구조 구현

Algorithm 1 PPO, Actor-Critic Style

for iteration$=1, 2, \ldots$ **do**
  for actor$=1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Rollout & GAE 계산 – 데이터 수집과 Advantage 추정

```
for step in range(0, config.num_steps):
    ...
    action, logprob, _, value = agent.get_action_and_value(next_obs)
    ...
    rewards[step] = torch.tensor(reward)
    ...
# GAE (Generalized Advantage Estimation)
with torch.no_grad():
    ...
    for t in reversed(range(config.num_steps)):
        …
        delta =     rewards[t] + config.gamma * nextvalues * nextnonterminal - values[t]
        advantages[t] = lastgaelam = delta + config.gamma * config.gae_lambda * nextnonterminal * lastgaelam
    returns = advantages + values
```

Clipped Surrogate Objective

```
logratio = newlogprob - b_logprobs[mb_inds]
ratio = logratio.exp()
pg_loss1 = -mb_advantages * ratio
pg_loss2 = -mb_advantages * torch.clamp(ratio, 1 - clip_coef, 1 + clip_coef)
pg_loss = torch.max(pg_loss1, pg_loss2).mean()
```

Value & Entropy 손실 + 최종 업데이트

```
# 가치 함수 손실 (Value Loss)
v_loss_unclipped = (newvalue - b_returns) ** 2
...
v_loss = 0.5 * torch.max(v_loss_unclipped, v_loss_clipped).mean()

# 엔트로피 손실 (탐험 유도)
entropy_loss = entropy.mean()

# 총 손실 함수
loss = pg_loss - ent_coef * entropy_loss + v_loss * vf_coef

optimizer.zero_grad()
loss.backward()
nn.utils.clip_grad_norm_(agent.parameters(), max_grad_norm)
optimizer.step()
```

2. 구현 및 소스코드 설명

1. 환경설정

```python
def make_env(env_id, seed, idx, run_name):
    def thunk():
        env = gym.make(env_id)
        env = gym.wrappers.RecordEpisodeStatistics(env)

        env = gym.wrappers.AtariPreprocessing(
            env,
            noop_max=30,
            frame_skip=4,
            screen_size=84,
            terminal_on_life_loss=True,
            grayscale_obs=True,
            grayscale_newaxis=False,
            scale_obs=True
        )
        env = gym.wrappers.FrameStack(env, 4)
        env.action_space.seed(seed)
        return env
    return thunk
```

- Pong환경을 초기화하고 학습에 적합한 환경으로 전처리
- frame_skip=4로 계산량을 감소
- FrameStack(env, 4)로 최근 4개의 프레임을 쌓아 시간에 따른 움직임의 방향, 속도를 알 수 있게 함
- grayscale로 변환
- scale_obs=True 값을 0~1로 정규화

2. 신경망 구성

```python
class Agent(nn.Module):
    def __init__(self, envs):
        super().__init__()
        self.network = nn.Sequential(
            nn.Conv2d(4, 32, 8, stride=4),
            nn.ReLU(),
            nn.Conv2d(32, 64, 4, stride=2),
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=1),
            nn.ReLU(),
            nn.Conv2d(128, 128, 3, stride=1),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(128 * 5 * 5, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU(),
        )
        self.actor = nn.Linear(256, envs.single_action_space.n)
        self.critic = nn.Linear(256, 1)

        self.apply(self._init_weights)
```

- CNN 기반의 actor-critic 신경망 구조를 사용
- FrameStack을 통해 쌓인 4개의 전 처리된 흑백 프레임을 입력으로 받아 4개의 Convolution layer를 거쳐 시각적 특징을 추출
- Fully Connected layer를 통해 256차원의 feature vector 생성
- 이를 각각 정책을 출력하는 actor head와 상태가치를 예측하는 critic head에 연결

3. 하이퍼 파라미터

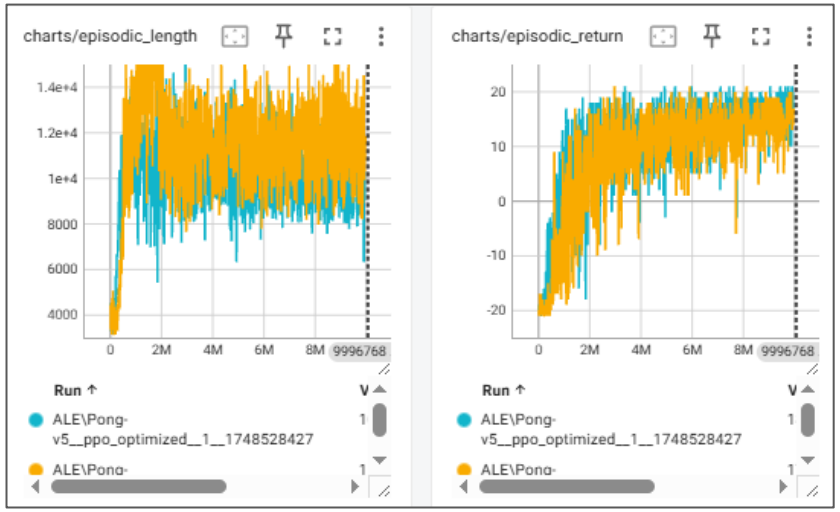
```python
total_timesteps = 10000000

learning_rate = 3e-4
num_envs = 16
num_steps = 256
anneal_lr = True
gamma = 0.99
gae_lambda = 0.95
num_minibatches = 8
update_epochs = 4
norm_adv = True
clip_coef = 0.2
ent_coef = 0.01
vf_coef = 0.5
max_grad_norm = 0.5
target_kl = None

batch_size = num_envs * num_steps
minibatch_size = batch_size // num_minibatches
num_updates = total_timesteps // batch_size
```

- clip_coef : 업데이트 비율을 제한
- num_envs : 학습 병렬환경
- num_steps * num_envs = batch_size
- batch_size를 minibatch size로 나눠 전체 묶음을 update epochs번 반복해서 학습

3. 학습결과

* 비슷하게 학습 결과가 나온 2개 비교

A 환경 param	A 환경 value	B 환경 param	B 환경 value
batch_size	8192	batch_size	4096
clip_coef	0.2	clip_coef	0.2
cuda	TRUE	cuda	TRUE
ent_coef	0.02	ent_coef	0.01
gae_lambda	0.95	gae_lambda	0.95
gamma	0.99	gamma	0.99
learning_rate	0.0005	learning_rate	0.0003
minibatch_size	512	minibatch_size	512
num_envs	16	num_envs	16
num_minibatches	16	num_minibatches	8
num_steps	512	num_steps	256
num_updates	1220	num_updates	2441
seed	1	seed	1
total_timesteps	10000000	total_timesteps	10000000
update_epochs	6	update_epochs	4
vf_coef	0.5	vf_coef	0.5

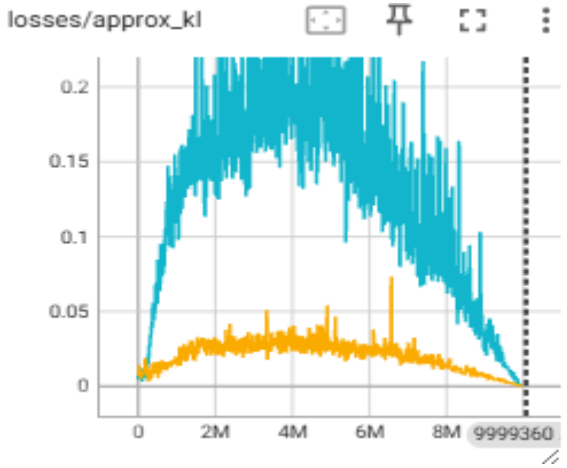


A, B 모두 에피소드당 최대 보상으로 수렴하는 모양

3. 학습결과

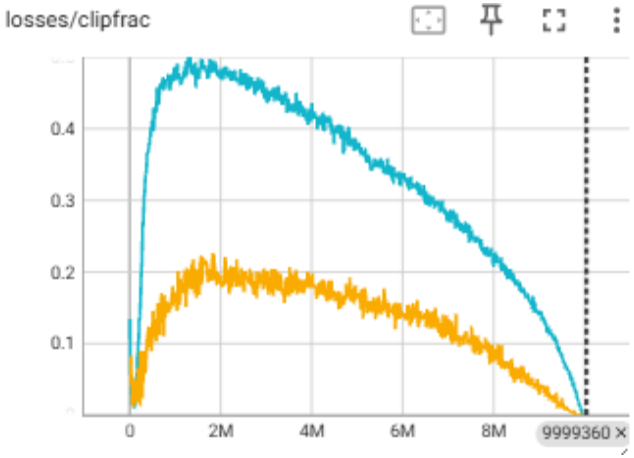
* 비슷하게 학습 결과가 나온 2개 비교 A 환경 B 환경

approx_kl



- B - KL발산이 낮아 정책이 너무 급격히 바뀌지 않는 것 확인
- A – 0.15 이상 상승했다 하락하여 정책이 너무 크게 업데이트 됨

clipfrac



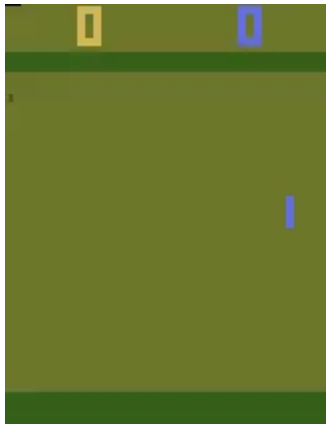
- B - clip이 전체적으로 0.2를 유지하여 과도한 제한 없이 학습이 이루어짐
- A – 과도한 클리핑

B 환경으로

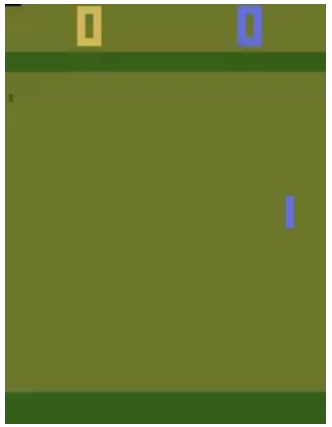
모델 테스트 진행

4. 실행결과

Reward : 20



Reward : 17



5. 프로그램 소스 - 학습코드

ppo_simple_config.py

```python
import os
import random
import time
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.distributions import Categorical
import gymnasium as gym
import numpy as np
from gymnasium.vector import SyncVectorEnv
from torch.utils.tensorboard import SummaryWriter
import cv2

class Config:
    exp_name = "ppo_simple_config"
    seed = 1
    torch_deterministic = True
    cuda = True
    save_model = True

    env_id = "ALE/Pong-v5"
    total_timesteps = 10000000

    learning_rate = 3e-4
    num_envs = 16
    num_steps = 256
    anneal_lr = True
    gamma = 0.99
    gae_lambda = 0.95
    num_minibatches = 8
    update_epochs = 4
    norm_adv = True
    clip_coef = 0.2
    ent_coef = 0.01
    vf_coef = 0.5
    max_grad_norm = 0.5
    target_kl = None

    batch_size = num_envs * num_steps
    minibatch_size = batch_size // num_minibatches
    num_updates = total_timesteps // batch_size

def make_env(env_id, seed, idx, run_name):
    def thunk():
        env = gym.make(env_id)
        env = gym.wrappers.RecordEpisodeStatistics(env)
```

```python
def make_env(env_id, seed, idx, run_name):
    def thunk():
        env = gym.wrappers.AtariPreprocessing(
            env,
            noop_max=30,
            frame_skip=4,
            screen_size=84,
            terminal_on_life_loss=True,
            grayscale_obs=True,
            grayscale_newaxis=False,
            scale_obs=True
        )
        env = gym.wrappers.FrameStack(env, 4)
        env.action_space.seed(seed)
        return env
    return thunk

class Agent(nn.Module):
    def __init__(self, envs):
        super().__init__()
        self.network = nn.Sequential(
            nn.Conv2d(4, 32, 8, stride=4),
            nn.ReLU(),
            nn.Conv2d(32, 64, 4, stride=2),
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=1),
            nn.ReLU(),
            nn.Conv2d(128, 128, 3, stride=1),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(128 * 5 * 5, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU(),
        )
        self.actor = nn.Linear(256, envs.single_action_space.n)
        self.critic = nn.Linear(256, 1)

        self.apply(self._init_weights)

    def _init_weights(self, m):
        if isinstance(m, nn.Linear):
            nn.init.orthogonal_(m.weight, gain=np.sqrt(2))
            nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.Conv2d):
            nn.init.orthogonal_(m.weight, gain=np.sqrt(2))
            nn.init.constant_(m.bias, 0)
```

```python
    def get_value(self, x):
        return self.critic(self.network(x))

    def get_action_and_value(self, x, action=None):
        hidden = self.network(x)
        logits = self.actor(hidden)
        probs = Categorical(logits=logits)
        if action is None:
            action = probs.sample()
        return action, probs.log_prob(action), probs.entropy(), self.critic(hidden)

if __name__ == "__main__":
    config = Config()
    run_name = f"{config.env_id}__{config.exp_name}__{config.seed}__{int(time.time())}"

    # TensorBoard 설정
    writer = SummaryWriter(f"runs/{run_name}")
    writer.add_text(
        "hyperparameters",
        "|param|value|\n|-|-|\n%s" % ("\n".join([f"|{key}|{getattr(config, key)}|" for key in dir(config) if not key.startswith('_')])),
    )

    # 시드 설정
    random.seed(config.seed)
    np.random.seed(config.seed)
    torch.manual_seed(config.seed)
    torch.backends.cudnn.deterministic = config.torch_deterministic

    device = torch.device("cuda" if torch.cuda.is_available() and config.cuda else "cpu")
    print(f"Using device: {device}")

    # 환경 설정
    envs = SyncVectorEnv(
        [make_env(config.env_id, config.seed + i, i, run_name) for i in range(config.num_envs)]
    )
    assert isinstance(envs.single_action_space, gym.spaces.Discrete), "only discrete action space is supported"

    agent = Agent(envs).to(device)
    optimizer = optim.AdamW(agent.parameters(), lr=config.learning_rate, eps=1e-5, weight_decay=1e-4)

    obs = torch.zeros((config.num_steps, config.num_envs, 4, 84, 84)).to(device)
    actions = torch.zeros((config.num_steps, config.num_envs) + envs.single_action_space.shape).to(device)
    logprobs = torch.zeros((config.num_steps, config.num_envs)).to(device)
    rewards = torch.zeros((config.num_steps, config.num_envs)).to(device)
    dones = torch.zeros((config.num_steps, config.num_envs)).to(device)
```

5. 프로그램 소스 - 학습코드

```python
if __name__ == "__main__":
    dones = torch.zeros((config.num_steps, config.num_envs)).to(device)
    values = torch.zeros((config.num_steps, config.num_envs)).to(device)

    # 게임 시작
    global_step = 0
    start_time = time.time()
    next_obs, _ = envs.reset(seed=config.seed)
    next_obs = torch.Tensor(next_obs).to(device)
    next_done = torch.zeros(config.num_envs).to(device)

    print(f"Starting PPO training for {config.total_timesteps:,} timesteps")
    print(f"TensorBoard: tensorboard --logdir=runs")
    print(f"Environment: {config.env_id}")
    print(f"Config: {config.num_envs} envs, {config.learning_rate} lr, {config.num_steps} steps")

    for iteration in range(1, config.num_updates + 1):

        if config.anneal_lr:
            frac = 1.0 - (iteration - 1.0) / config.num_updates
            lrnow = frac * config.learning_rate
            optimizer.param_groups[0]["lr"] = lrnow

        for step in range(0, config.num_steps):
            global_step += config.num_envs
            obs[step] = next_obs
            dones[step] = next_done

            with torch.no_grad():
                action, logprob, _, value = agent.get_action_and_value(next_obs)
                values[step] = value.flatten()
            actions[step] = action
            logprobs[step] = logprob

            next_obs, reward, terminations, truncations, infos = envs.step(action.cpu().numpy())
            next_done = np.logical_or(terminations, truncations)
            rewards[step] = torch.tensor(reward).to(device).view(-1)

            next_obs = torch.Tensor(next_obs).to(device)
            next_done = torch.Tensor(next_done).to(device)

            if "final_info" in infos:
                for info in infos["final_info"]:
                    if info and "episode" in info:
                        episode_reward = info['episode']['r']
                        if step % 1000 == 0:
                            print(f"Step {global_step:,}: Episode reward = {float(episode_reward):.1f}")
                        writer.add_scalar("charts/episodic_return", episode_reward, global_step)
```

```python
                        writer.add_scalar("charts/episodic_length", info["episode"]["l"], global_step)
        with torch.no_grad():
            next_value = agent.get_value(next_obs).reshape(1, -1)
            advantages = torch.zeros_like(rewards).to(device)
            lastgaelam = 0
            for t in reversed(range(config.num_steps)):
                if t == config.num_steps - 1:
                    nextnonterminal = 1.0 - next_done
                    nextvalues = next_value
                else:
                    nextnonterminal = 1.0 - dones[t + 1]
                    nextvalues = values[t + 1]
                delta = rewards[t] + config.gamma * nextvalues * nextnonterminal - values[t]
                advantages[t] = lastgaelam = delta + config.gamma * config.gae_lambda * nextnonterminal * lastgaelam
            returns = advantages + values

        b_obs = obs.reshape((-1,) + (4, 84, 84))
        b_logprobs = logprobs.reshape(-1)
        b_actions = actions.reshape((-1,) + envs.single_action_space.shape)
        b_advantages = advantages.reshape(-1)
        b_returns = returns.reshape(-1)
        b_values = values.reshape(-1)

        b_inds = np.arange(config.batch_size)
        clipfracs = []
        for epoch in range(config.update_epochs):
            np.random.shuffle(b_inds)
            for start in range(0, config.batch_size, config.minibatch_size):
                end = start + config.minibatch_size
                mb_inds = b_inds[start:end]

                _, newlogprob, entropy, newvalue = agent.get_action_and_value(b_obs[mb_inds], b_actions.long()[mb_inds])
                logratio = newlogprob - b_logprobs[mb_inds]
                ratio = logratio.exp()

                with torch.no_grad():
                    old_approx_kl = (-logratio).mean()
                    approx_kl = ((ratio - 1) - logratio).mean()
                    clipfracs += [((ratio - 1.0).abs() > config.clip_coef).float().mean().item()]

                mb_advantages = b_advantages[mb_inds]
                if config.norm_adv:
```

5. 프로그램 소스 - 학습코드

```python
            mb_advantages = (mb_advantages - mb_advantages.mean()) / (mb_advantages.std() + 1e-8)

        #Clip 정책 손실
        pg_loss1 = -mb_advantages * ratio
        pg_loss2 = -mb_advantages * torch.clamp(ratio, 1 - config.clip_coef, 1 + config.clip_coef)
        pg_loss = torch.max(pg_loss1, pg_loss2).mean()


        newvalue = newvalue.view(-1)
        if config.clip_coef:
            v_loss_unclipped = (newvalue - b_returns[mb_inds]) ** 2
            v_clipped = b_values[mb_inds] + torch.clamp(
                newvalue - b_values[mb_inds],
                -config.clip_coef,
                config.clip_coef,
            )
            v_loss_clipped = (v_clipped - b_returns[mb_inds]) ** 2
            v_loss_max = torch.max(v_loss_unclipped, v_loss_clipped)
            v_loss = 0.5 * v_loss_max.mean()
        else:
            v_loss = 0.5 * ((newvalue - b_returns[mb_inds]) ** 2).mean()

        entropy_loss = entropy.mean()
        loss = pg_loss - config.ent_coef * entropy_loss + v_loss * config.vf_coef

        optimizer.zero_grad()
        loss.backward()
        nn.utils.clip_grad_norm_(agent.parameters(), config.max_grad_norm)
        optimizer.step()

    if config.target_kl is not None and approx_kl > config.target_kl:
        break

y_pred, y_true = b_values.cpu().numpy(), b_returns.cpu().numpy()
var_y = np.var(y_true)
explained_var = np.nan if var_y == 0 else 1 - np.var(y_true - y_pred) / var_y

# 로깅
writer.add_scalar("charts/learning_rate", optimizer.param_groups[0]["lr"], global_step)
writer.add_scalar("losses/value_loss", v_loss.item(), global_step)
writer.add_scalar("losses/policy_loss", pg_loss.item(), global_step)
writer.add_scalar("losses/entropy", entropy_loss.item(), global_step)
```

```python
    writer.add_scalar("losses/old_approx_kl", old_approx_kl.item(), global_step)
    writer.add_scalar("losses/approx_kl", approx_kl.item(), global_step)
    writer.add_scalar("losses/clipfrac", np.mean(clipfracs), global_step)
    writer.add_scalar("losses/explained_variance", explained_var, global_step)

    # 진행 상황 출력
    if iteration % 50 == 0:
        sps = int(global_step / (time.time() - start_time))
        print(f"Iteration {iteration:,}/{config.num_updates:,} | SPS: {sps} | Step: {global_step:,}")

    writer.add_scalar("charts/SPS", int(global_step / (time.time() - start_time)), global_step)


model_dir = f"runs/{run_name}"
os.makedirs(model_dir, exist_ok=True)
model_path = f"{model_dir}/{config.exp_name}.cleanrl_model"
torch.save(agent.state_dict(), model_path)
print(f"Model saved to {model_path}")

# 저장
if os.path.exists(model_path):
    print(f"Model file confirmed: {os.path.getsize(model_path)} bytes")
else:
    print(f"Model save failed!")

envs.close()
writer.close()
```

```
Iteration 1,300/2,441 | SPS: 983 | Step: 5,324,800
Iteration 1,350/2,441 | SPS: 982 | Step: 5,529,600
Iteration 1,400/2,441 | SPS: 982 | Step: 5,734,400
Iteration 1,450/2,441 | SPS: 982 | Step: 5,939,200
Iteration 1,500/2,441 | SPS: 982 | Step: 6,144,000
Iteration 1,550/2,441 | SPS: 981 | Step: 6,348,800
Iteration 1,600/2,441 | SPS: 981 | Step: 6,553,600
Iteration 1,650/2,441 | SPS: 981 | Step: 6,758,400
Step 6,848,528: Episode reward = 14.0
Iteration 1,700/2,441 | SPS: 981 | Step: 6,963,200
Iteration 1,750/2,441 | SPS: 981 | Step: 7,168,000
Iteration 1,800/2,441 | SPS: 981 | Step: 7,372,800
Step 7,389,200: Episode reward = 15.0
Iteration 1,850/2,441 | SPS: 980 | Step: 7,577,600
Iteration 1,900/2,441 | SPS: 980 | Step: 7,782,400
Iteration 1,950/2,441 | SPS: 980 | Step: 7,987,200
Iteration 2,000/2,441 | SPS: 980 | Step: 8,192,000
Iteration 2,050/2,441 | SPS: 979 | Step: 8,396,800
Step 8,409,104: Episode reward = 19.0
Iteration 2,100/2,441 | SPS: 979 | Step: 8,601,600
Iteration 2,150/2,441 | SPS: 979 | Step: 8,806,400
Iteration 2,200/2,441 | SPS: 979 | Step: 9,011,200
Step 9,121,808: Episode reward = 17.0
Iteration 2,250/2,441 | SPS: 979 | Step: 9,216,000
Step 9,367,568: Episode reward = 19.0
Iteration 2,300/2,441 | SPS: 979 | Step: 9,420,800
Iteration 2,350/2,441 | SPS: 979 | Step: 9,625,600
Iteration 2,400/2,441 | SPS: 979 | Step: 9,830,400
Step 9,941,008: Episode reward = 14.0
```

5. 프로그램 소스 – 테스트코드

*5번 실행

```python
import os
import random
import time
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.distributions import Categorical
import gymnasium as gym
import numpy as np

MODEL_PATH = "runs/ALE/Pong-v5__ppo_optimized__1__1748580656_000/ppo_optimized.cleanrl_model"
ENV_ID = "ALE/Pong-v5"
SEED = 1
NUM_EPISODES = 5
RENDER = False
SAVE_VIDEO = True
VIDEO_FOLDER = "test_videos"

def make_env(env_id, seed, save_video=False, video_folder="test_videos"):
    if save_video:
        env = gym.make(env_id, render_mode="rgb_array", frameskip=1)
        os.makedirs(video_folder, exist_ok=True)
        env = gym.wrappers.RecordVideo(
            env,
            video_folder,
            episode_trigger=lambda x: True,
            name_prefix="ppo_test"
        )
    else:

        env = gym.make(env_id, render_mode="human" if RENDER else None, frameskip=1)

    env = gym.wrappers.RecordEpisodeStatistics(env)

    env = gym.wrappers.AtariPreprocessing(
        env,
        noop_max=30,
        frame_skip=4,
        screen_size=84,
        terminal_on_life_loss=True,
        grayscale_obs=True,
        grayscale_newaxis=False,
        scale_obs=True
    )

    env = gym.wrappers.FrameStack(env, 4)
    env.action_space.seed(seed)
    return env
```

```python
class Agent(nn.Module):
    def __init__(self, action_space_n):
        super().__init__()

        self.network = nn.Sequential(
            nn.Conv2d(4, 32, 8, stride=4),
            nn.ReLU(),
            nn.Conv2d(32, 64, 4, stride=2),
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=1),
            nn.ReLU(),
            nn.Conv2d(128, 128, 3, stride=1),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(128 * 5 * 5, 512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.ReLU(),
        )
        self.actor = nn.Linear(256, action_space_n)
        self.critic = nn.Linear(256, 1)

    def get_action(self, x, deterministic=False):
        hidden = self.network(x)
        logits = self.actor(hidden)
        probs = Categorical(logits=logits)
        if deterministic:
            action = torch.argmax(logits, dim=-1)
        else:
            action = probs.sample()
        return action

if __name__ == "__main__":
    print("🎮 PPO 모델 테스트 시작!")
    print(f"📁 모델 경로: {MODEL_PATH}")
    print(f"🎯 환경: {ENV_ID}")
    print(f"🔢 에피소드 수: {NUM_EPISODES}")
    print("-" * 50)

    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"디바이스: {device}")

    env = make_env(ENV_ID, SEED, save_video=SAVE_VIDEO, video_folder=VIDEO_FOLDER)

    agent = Agent(env.action_space.n).to(device)

    try:
        agent.load_state_dict(torch.load(MODEL_PATH, map_location=device))
        agent.eval()
    except Exception as e:
        print(f"모델 로드 실패: {e}")
        exit(1)

    if SAVE_VIDEO:
        print(f"동영상이 '{VIDEO_FOLDER}' 폴더에 저장")
```

```python
    # 테스트 실행
    episode_rewards = []
    episode_lengths = []

    for episode in range(NUM_EPISODES):
        obs, _ = env.reset(seed=SEED + episode)
        episode_reward = 0
        episode_length = 0
        done = False

        while not done:
            obs_tensor = torch.tensor(obs).unsqueeze(0).to(device)

            with torch.no_grad():
                action = agent.get_action(obs_tensor, deterministic=True)

            obs, reward, terminated, truncated, info = env.step(action.cpu().numpy().item())
            done = terminated or truncated

            episode_reward += reward
            episode_length += 1

            if RENDER and not SAVE_VIDEO:
                env.render()
                time.sleep(0.02)

        episode_rewards.append(episode_reward)
        episode_lengths.append(episode_length)

        print(f" 에피소드 {episode + 1} 완료")
        print(f"   보상: {episode_reward:.1f}")
        print(f"   길이: {episode_length} 스텝")

    if SAVE_VIDEO:
        print(f"저장완료")

    env.close()
```

```
 에피소드 2 완료
   보상: 20.0
   길이: 2845 스텝
Moviepy - Building video C:\project\pingpong_ppo\test_videos\ppo_test-episode-2.mp4.
Moviepy - Writing video C:\project\pingpong_ppo\test_videos\ppo_test-episode-2.mp4

Moviepy - Done !
Moviepy - video ready C:\project\pingpong_ppo\test_videos\ppo_test-episode-2.mp4
 에피소드 3 완료
   보상: 15.0
   길이: 3009 스텝
Moviepy - Building video C:\project\pingpong_ppo\test_videos\ppo_test-episode-3.mp4.
Moviepy - Writing video C:\project\pingpong_ppo\test_videos\ppo_test-episode-3.mp4

Moviepy - Done !
Moviepy - video ready C:\project\pingpong_ppo\test_videos\ppo_test-episode-3.mp4
 에피소드 4 완료
   보상: 11.0
   길이: 3577 스텝
Moviepy - Building video C:\project\pingpong_ppo\test_videos\ppo_test-episode-4.mp4.
Moviepy - Writing video C:\project\pingpong_ppo\test_videos\ppo_test-episode-4.mp4

Moviepy - Done !
Moviepy - video ready C:\project\pingpong_ppo\test_videos\ppo_test-episode-4.mp4
 에피소드 5 완료
   보상: 17.0
   길이: 2868 스텝

Moviepy - Done !
Moviepy - video ready C:\project\pingpong_ppo\test_videos\ppo_test-episode-4.mp4
 에피소드 5 완료
   보상: 17.0
   길이: 2868 스텝
Moviepy - Done !
Moviepy - video ready C:\project\pingpong_ppo\test_videos\ppo_test-episode-4.mp4
 에피소드 5 완료
   보상: 17.0
   길이: 2868 스텝
Moviepy - video ready C:\project\pingpong_ppo\test_videos\ppo_test-episode-4.mp4
 에피소드 5 완료
   보상: 17.0
   길이: 2868 스텝
 에피소드 5 완료
   보상: 17.0
   길이: 2868 스텝
   보상: 17.0
   길이: 2868 스텝
   길이: 2868 스텝
저장완료
```

감사합니다